

REAL-TIME PATIENT MONITORING SYSTEM

Development on Raspberry Pi 5

Prepared by: Masrur Jamil Prochchhod

Institution: Hochschule Hamm-Lippstadt (HSHL)

Platform: Raspberry Pi 5 | Python 3 | Blynk IoT

PROJECT OVERVIEW

Goal

Build a Raspberry Pi 5 based patient monitoring prototype with local hardware output and cloud sync.

Sensors (Simulated)

Heart Rate
Body Temperature
Physical Acceleration

All generated via sim.py — no physical sensors wired in this phase.

Real Hardware

HD44780 16×2 LCD
(GPIO, 4-bit mode)

Active Piezo Buzzer
(GPIO Pin 17)

HARDWARE SETUP — Raspberry Pi 5 + GPIO

HD44780 16×2 LCD — 4-Bit Parallel Mode

RS	GPIO 22
E (Enable)	GPIO 18
D4	GPIO 25
D5	GPIO 24
D6	GPIO 23
D7	GPIO 12

Active Piezo Buzzer

GPIO Pin 17 | Direct Digital Drive (HIGH/LOW)
No PWM required — triggers instantly on threshold breach

Simulated Vital Parameters

Heart Rate

Range: 40 – 150 BPM

Tachycardia / cardiac monitoring

Body Temperature

Range: 35°C – 42°C

Fever / hypothermia detection

Acceleration (3-axis)

Range: 0 – 5 G

Fall detection / motion monitoring

THE HARDWARE STRUGGLE — SD Card Boot Failure

⚠ The Pink Screen

Every boot attempt produced:

- Solid pink / rainbow screen
- Error messages then halt
- Occasional partial boot → freeze
- Rare successful boots that wouldn't repeat

Days passed with no usable system.

Possible Causes Investigated:

Corrupted SD filesystem	Bootloader cannot read boot partition
Degraded SD sectors	Physical wear causing intermittent failures
Incompatible card speed	Card too slow for Pi 5's I/O demands
Underpowered supply	Pi 5 draws more current than earlier models
Bad OS image write	Initial flash incomplete or corrupted

✓ Resolution

After exhausting all other options, the SD card was replaced with a new high-quality Class 10 / U3 card. The Pi 5 booted cleanly and consistently on every subsequent power cycle — immediately and definitively.

Key lesson: SD cards are a common point of failure on Pi 5. Use a Class 10/U3 card or NVMe SSD from the outset.

SOFTWARE CHALLENGE A — Wayland Display Driver Conflict

Symptom

matplotlib plot window opened completely blank — or threw a display server exception immediately on launch. No plot content rendered at all.

Root Cause

RPi 5 migrated from X11 → Wayland desktop protocol. matplotlib defaulted to the QtAgg backend, which tried to open via X11. On a Wayland system, QT_QPA_PLATFORM mismatch caused the rendering pipeline to fail entirely.

Resolution

Force-switch matplotlib to the TkAgg backend before any other import. Tkinter canvas renders correctly under Wayland without the Qt/X11 dependency.

The Fix — one line before all other matplotlib code:

```
import matplotlib
matplotlib.use('TkAgg') # Must be called BEFORE importing matplotlib.pyplot
import matplotlib.pyplot as plt
```

SOFTWARE CHALLENGE B — CSV Race Conditions & Memory Leaks

Problem 1: File Zeroing Race

sim.py used write mode ('w') — erasing the entire CSV on every loop. Reading scripts caught the file at 0 bytes, causing flat lines, index errors, and crashes.

Problem 2: Memory Leak

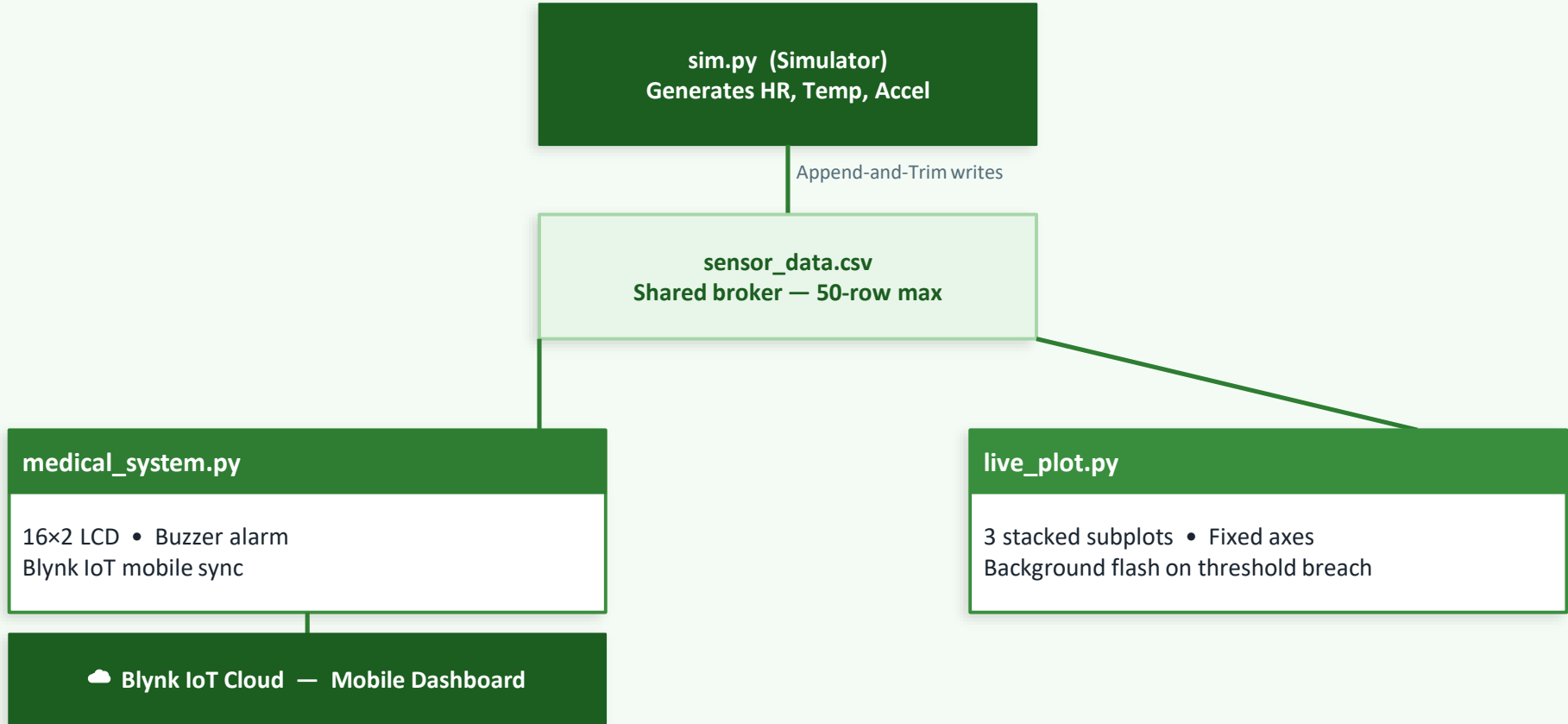
FuncAnimation appended data arrays on every frame with no size limit. Arrays grew indefinitely, consuming RAM until the system slowed and failed.

✓ Resolution: Append-and-Trim Pattern

```
# WRITE: append only — never overwrites the file
with open(FILENAME, 'a', newline='') as f:
    writer.writerow([hr, tmp, acc, t_now, d_now])

# READ: moving window — discards corrupted frames
try:
    lines = list(csv.reader(f))
    if len(lines) < 2: return
    recent = lines[
-20:] # last 20 rows only
except (ValueError, IndexError, csv.Error): pass
```

FINAL SYSTEM ARCHITECTURE — Data Flow



LIVE VISUALIZATION — Threshold Monitoring

Heart Rate

Axis Range: 40 – 150 BPM

Alarm Threshold: 100 BPM

Alert: Background → red

```
ax1.set_facecolor('#ffcccc' if hrs[-1] > 100  
else 'white')
```

Body Temperature

Axis Range: 35°C – 42°C

Alarm Threshold: 38.5°C

Alert: Background → red

```
ax2.set_facecolor('#ffcccc' if tmps[-1] >  
38.5 else 'white')
```

Acceleration

Axis Range: 0 – 5 G

Alarm Threshold: 2.0 G

Alert: Background → red

```
ax3.set_facecolor('#ffcccc' if accs[-1] >  
2.0 else 'white')
```

Design principle: Fixed axis ranges (not auto-scale) prevent clinical variations from being visually compressed and missed. Each subplot also shows a dashed horizontal threshold line as a persistent reference even when values are within normal range.

CONCLUSION & NEXT STEPS

Challenges Resolved:

- ✓ [Hardware] SD card pink screen — replaced card
- ✓ [Software] Wayland/Qt blank plot — TkAgg fix
- ✓ [Software] CSV race condition — append-and-trim
- ✓ [Software] Memory leak — moving-window reads
- ✓ [Software] Flat line rendering — defensive parsing

Next Steps:

- Wire real sensors: MAX30102, DS18B20, MPU6050
- Migrate to NVMe SSD for reliable storage
- Replace CSV broker with MQTT messaging
- Add multi-patient support & identity layer
- Deploy on edge device with UPS backup